

MÓDULO 2: OBTENCIÓN Y CARGA DE DATOS

Proyecto: Análisis ETL - Entidad Financiera

Prerequisito: MÓDULO 1 completado (entorno configurado)

OBJETIVO

Obtener los datasets necesarios, cargarlos en Python usando Pandas y realizar una exploración inicial para entender la estructura de los datos.

PASO 1: OBTENER LOS DATASETS

► 1.1 Datasets del proyecto

Para este proyecto necesitamos **2 archivos CSV**:

1. `customer_data.csv` - Datos de clientes
2. `sales_data.csv` - Datos de transacciones

Fuente: Istanbul Shopping Customer Data (Kaggle o proporcionado por el profesor)

► 1.2 Ubicar archivos en el proyecto

```
# Copiar los CSV a la carpeta datos/:  
# Opción 1: Arrastrar archivos a la carpeta desde explorador de archivos  
# Opción 2: Usar comando copy en PowerShell  
  
copy "C:\ruta\origen\customer_data.csv" ".\datos\  
copy "C:\ruta\origen\sales_data.csv" ".\datos\  

```

Verificar:

```
ls .\datos\  

```

Salida esperada:

```
Directory: C:\...\ABP INNOVACION\datos  
  
Mode                LastWriteTime         Length Name  
----                -  
-a---             10/10/2025   09:15         2734567 customer_data.csv  
-a---             10/10/2025   09:15         5987234 sales_data.csv  

```



PASO 2: CREAR EL NOTEBOOK JUPYTER

► 2.1 ¿Qué es un Notebook Jupyter?

Documento interactivo que combina:

- **Código Python** ejecutable por celdas
- **Texto explicativo** en formato Markdown
- **Visualizaciones** (gráficos, tablas)
- **Resultados** de ejecución

Ventajas:

- Documentación integrada con código
 - Ejecución paso a paso
 - Ideal para análisis exploratorio
-

► 2.2 Crear notebook desde VS Code

```
# Abrir VS Code en la carpeta del proyecto:  
code .
```

Pasos en VS Code:

1. `Ctrl + Shift + P` → Abrir paleta de comandos
2. Escribir: `Jupyter: Create New Blank Notebook`
3. Guardar como: `notebooks/analisis_etl.ipynb`

Alternativa desde terminal:

```
# Crear archivo directamente:  
New-Item -Path ".\notebooks\analisis_etl.ipynb" -ItemType File
```

► 2.3 Configurar kernel de Python

¿Qué es un kernel? Motor que ejecuta el código del notebook.

Pasos:

1. Abrir `analisis_etl.ipynb` en VS Code
 2. Arriba a la derecha verás "Select Kernel"
 3. Elegir: `Python 3.11.x ('venv': venv)`
 4. Si no aparece, seleccionar "Python Environments" y buscar `.\venv\Scripts\python.exe`
-



PASO 3: PRIMERA CELDA - IMPORTAR LIBRERÍAS

► 3.1 Crear celda Markdown de título

Clic en “+ Markdown” arriba del notebook.

Contenido:

```
#  ANÁLISIS ETL - MEDIOS DE PAGO
## Trabajo Práctico - TSCDIA 2025

**Equipo:**
- Paola Garcia
- Pablo Taborda
- Julio Orjindo
- Rodenas Elias Gabriel

**Objetivo:** Realizar proceso ETL completo sobre datos de clientes y ventas
```

► 3.2 Crear celda de código para imports

Clic en “+ Code”

Código:

```
# Importar librerías necesarias para el análisis
import pandas as pd          # Manipulación de datos
import numpy as np           # Operaciones numéricas
import matplotlib.pyplot as plt # Visualizaciones
import os                    # Operaciones del sistema operativo
from datetime import datetime # Manejo de fechas

# Configuración de visualizaciones
plt.style.use('default')
plt.rcParams['figure.figsize'] = (12, 6) # Tamaño por defecto de gráficos
plt.rcParams['font.size'] = 10

print('☑ Librerías importadas correctamente')
print(f'Versión de Pandas: {pd.__version__}')
print(f'Versión de NumPy: {np.__version__}')
```

Ejecutar celda: **Shift + Enter**

Salida esperada:

```
☑ Librerías importadas correctamente
Versión de Pandas: 2.1.1
Versión de NumPy: 1.26.0
```



PASO 4: CARGAR DATASETS

► 4.1 Celda para cargar customer_data.csv

Nueva celda de código:

```
# PASO 1: CARGAR DATOS DE CLIENTES
print('=' * 80)
print('CARGANDO DATOS DE CLIENTES')
print('=' * 80)

# Leer archivo CSV
df_customers = pd.read_csv('../customer_data.csv')

# Mostrar información básica
print(f'\n✅ Archivo cargado exitosamente')
print(f'📊 Dimensiones: {df_customers.shape[0]} filas x {df_customers.shape[1]} columnas')
print(f'\n📋 Columnas encontradas:')
print(df_customers.columns.tolist())
```

¿Por qué `../customer_data.csv` ?

- El notebook está en `notebooks/`
- El CSV está en `datos/`
- `..` significa "subir un nivel" (salir de notebooks/)
- Luego busca en `datos/`

Ruta completa: `notebooks/../datos/customer_data.csv` = `datos/customer_data.csv`

Salida esperada:

```
=====
CARGANDO DATOS DE CLIENTES
=====

✅ Archivo cargado exitosamente
📊 Dimensiones: 99457 filas x 5 columnas

📋 Columnas encontradas:
['customer_id', 'age', 'gender', 'location', 'membership_years']
```

► 4.2 Explorar primeras filas

Nueva celda:

```
# Ver primeras 5 filas
print('\n🔗 PRIMERAS 5 FILAS DEL DATASET DE CLIENTES:')
print(df_customers.head())
```

Salida (ejemplo):

	customer_id	age	gender	location	membership_years
0	1001	34	Female	New York	5
1	1002	28	Male	Chicago	3
2	1003	45	Female	Miami	8
3	1004	52	Male	Los Angeles	12
4	1005	29	Female	New York	2

► 4.3 Cargar sales_data.csv

Nueva celda:

```
# PASO 2: CARGAR DATOS DE VENTAS
print('=' * 80)
print('CARGANDO DATOS DE VENTAS')
print('=' * 80)

df_sales = pd.read_csv('../sales_data.csv')

print(f'\n✅ Archivo cargado exitosamente')
print(f'📊 Dimensiones: {df_sales.shape[0]} filas x {df_sales.shape[1]} columnas')
print(f'\n📋 Columnas encontradas:')
print(df_sales.columns.tolist())

# Ver primeras filas
print('\n🔗 PRIMERAS 5 FILAS DEL DATASET DE VENTAS:')
print(df_sales.head())
```

Salida esperada:

```
=====
CARGANDO DATOS DE VENTAS
=====
```

☒ Archivo cargado exitosamente

 Dimensiones: 99457 filas x 8 columnas

 Columnas encontradas:

```
['transaction_id', 'customer_id', 'invoice_date', 'invoice_no', 'product_category',
 'quantity', 'price', 'payment_method', 'shopping_mall']
```

 PRIMERAS 5 FILAS DEL DATASET DE VENTAS:

	transaction_id	customer_id	invoice_date	invoice_no	product_category	quantity	price	payment_method	shopping_mall
0	1	1001	15-01-2021	I001	Clothing	2	50.0	Credit Card	Shopping Mall
1	2	1002	15-01-2021	I002	Electronics	1	120.0	Credit Card	Shopping Mall
...									

PASO 5: EXPLORACIÓN INICIAL DE DATOS

► 5.1 Información general de los DataFrames

Nueva celda:

```
# ANÁLISIS DE ESTRUCTURA DE DATOS
print('=' * 80)
print('INFORMACIÓN DETALLADA - DATASET DE CLIENTES')
print('=' * 80)

df_customers.info()
```

¿Qué muestra `.info()` ?

- Número de filas y columnas
- Nombre de cada columna
- Tipo de dato (`int64` , `float64` , `object`)
- Valores no nulos por columna
- Uso de memoria

Salida (ejemplo):

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99457 entries, 0 to 99456
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customer_id           99457 non-null  int64
1   age                   99338 non-null  float64
2   gender                99457 non-null  object
3   location              99457 non-null  object
4   membership_years      99457 non-null  int64
dtypes: float64(1), int64(2), object(2)
memory usage: 3.8+ MB
```

Observación crítica:

- `age` tiene **119 valores nulos** ($99457 - 99338 = 119$)
- `age` es `float64` (debería ser `int64`, pero tiene nulos)

► 5.2 Estadísticas descriptivas

Nueva celda:

```
# Estadísticas de columnas numéricas
print('\n📊 ESTADÍSTICAS DESCRIPTIVAS - CLIENTES:')
print(df_customers.describe())
```

¿Qué muestra `.describe()` ?

- `count` : cantidad de valores no nulos
- `mean` : promedio
- `std` : desviación estándar
- `min/max` : valores mínimo y máximo
- `25%, 50%, 75%` : percentiles

Salida (ejemplo):

	customer_id	age	membership_years
count	99457.000000	99338.000000	99457.000000
mean	50729.000000	44.234567	10.123456
std	28713.456789	12.345678	5.678901
min	1.000000	18.000000	0.000000
25%	25365.000000	35.000000	6.000000
50%	50729.000000	44.000000	10.000000
75%	76093.000000	53.000000	14.000000
max	99457.000000	70.000000	20.000000

Interpretación:

- Edad promedio: ~44 años
- Rango de edad: 18 a 70 años
- Membresía promedio: ~10 años

► 5.3 Valores únicos en columnas categóricas

Nueva celda:

```
# ANÁLISIS DE VARIABLES CATEGÓRICAS
print('\n📄 VALORES ÚNICOS POR COLUMNA:')
print(f'Géneros: {df_customers["gender"].unique()}')
print(f'Ubicaciones: {df_customers["location"].unique()}')
print(f'\n📊 FRECUENCIA DE GÉNEROS:')
print(df_customers["gender"].value_counts())
print(f'\n📊 FRECUENCIA DE UBICACIONES:')
print(df_customers["location"].value_counts())
```

Salida (ejemplo):

 VALORES ÚNICOS POR COLUMNA:

Géneros: ['Female' 'Male']

Ubicaciones: ['New York' 'Chicago' 'Miami' 'Los Angeles']

 FRECUENCIA DE GÉNEROS:

Female 59412

Male 40045

Name: gender, dtype: int64

 FRECUENCIA DE UBICACIONES:

Los Angeles 25035

Miami 24952

Chicago 24727

New York 24743

Name: location, dtype: int64



PASO 6: GUARDAR PROGRESO

► 6.1 Guardar notebook

Ctrl + S o File > Save

► 6.2 Commit a Git

```
# Agregar notebook al repositorio:
git add notebooks/analisis_etl.ipynb

# Commit con mensaje descriptivo:
git commit -m "Agregar notebook con carga inicial de datos"

# Subir a GitHub:
git push origin main
```



RESULTADO ESPERADO AL FINAL DEL MÓDULO 2

► Datos cargados exitosamente:

- ☒ `df_customers` : 99,457 registros × 5 columnas
- ☒ `df_sales` : 99,457 registros × 8 columnas

► Exploración completada:

- ☒ Tipos de datos identificados
 - ☒ Valores nulos detectados (119 en edad)
 - ☒ Rango de valores conocido
 - ☒ Distribución de categorías visualizada
-



COMANDOS PANDAS APRENDIDOS

Comando	Función
<code>pd.read_csv('archivo.csv')</code>	Cargar archivo CSV
<code>df.head()</code>	Mostrar primeras 5 filas
<code>df.tail()</code>	Mostrar últimas 5 filas
<code>df.shape</code>	Dimensiones (filas, columnas)
<code>df.columns</code>	Lista de columnas
<code>df.info()</code>	Información de estructura
<code>df.describe()</code>	Estadísticas descriptivas
<code>df['columna'].unique()</code>	Valores únicos
<code>df['columna'].value_counts()</code>	Frecuencia de valores



PRÓXIMOS PASOS

Ver **MÓDULO 3: LIMPIEZA Y TRANSFORMACIÓN DE DATOS (ETL)** para:

- Detectar y manejar valores nulos
- Transformar tipos de datos
- Crear nuevas columnas calculadas
- Fusionar los dos DataFrames

Documento creado: 8 de noviembre de 2025

Parte de: Documentación Técnica Completa - Proyecto ETL TSCDIA